

Python E-Course

Module 8 — Sharpening

Detailed Notes

Module Overview

Level: Filtering

Duration: ~1 week

Prerequisites: Modules 1-7

What You'll Learn

- Sharpen with the Laplacian and custom kernels.
- Apply unsharp masking (the photographer's trick).
- Understand sharpening = original + amount * (original - blurred).
- Use frequency-domain intuition for high-pass filters.

Lessons

#	Lesson	Duration
1	Why Sharpen?	20 min
2	Laplacian Kernel	30 min
3	Unsharp Masking	30 min
4	High-Pass Filtering	25 min
5	Custom Sharpening Kernels	25 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 9: Edge Detection

Lesson 1: Why Sharpen?

Module: 8 — Sharpening

Duration: 20 minutes

Difficulty: Beginner

Learning Objectives

- Explain what "sharpness" means at the pixel level.
- Identify situations where sharpening helps and where it hurts.
- Predict noise amplification from naive sharpening.

Prerequisites

- Module 7

1. Why This Matters

Sharpening is the photographer's everyday tool — slightly soft images become crisp; out-of-focus regions appear focused (within limits). But sharpening also amplifies noise, so understanding the trade-off matters.

2. Concept

2.1 What sharpness is

Sharpness = high local contrast at edges. A sharp image has steep transitions; a blurry one has gradual ramps. Mathematically, the derivative (or gradient) is large at sharp edges and small in flat regions.

2.2 The core idea

Almost every sharpening method boils down to:

$$\text{sharpened} = \text{original} + \text{amount} \times (\text{high_pass_of_original})$$

Where "high pass" emphasises the rapid-changing parts (edges) and suppresses the smooth parts. Add a scaled high-pass back to the original → edges become exaggerated.

Equivalently:

$$\text{sharpened} = \text{original} + \text{amount} \times (\text{original} - \text{blurred})$$

The blurred image has the high frequencies removed; subtracting recovers them.

2.3 When sharpening helps

- Slightly soft photos (camera focus close but not perfect).
- Down-scaled images (scaling tends to soften).
- Pre-processing before OCR or QR-code detection.

2.4 When sharpening hurts

- Already-sharp images → ringing / halos.
- Noisy images → noise becomes more visible.
- High-ISO photos → grain pops out.

2.5 Halos and ringing

Over-sharpening creates bright/dark halos around edges. This is the easiest way to spot too much sharpening.

3. Code Examples

Example 1: A "soft" image vs a "sharp" image

```
import cv2
from skimage.data import camera

img = camera()
blur = cv2.GaussianBlur(img, (0, 0), 2.0) # simulate softening

# Print pixel-derivative magnitude (a proxy for sharpness)
import numpy as np
def edge_energy(x):
    gx = np.gradient(x.astype(np.float32), axis=1)
    gy = np.gradient(x.astype(np.float32), axis=0)
    return float(np.sqrt(gx**2 + gy**2).mean())

print("sharp original :", edge_energy(img))
print("blurred copy   :", edge_energy(blur))
```

Expected Output:

```
sharp original : 13.4
blurred copy   : 5.8
```

The blurred copy has less than half the average gradient magnitude — quantitatively softer.

Example 2: A sharpening preview (covered in detail in next lessons)

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
blur = cv2.GaussianBlur(img, (0, 0), 1.5)
sharp = cv2.addWeighted(img, 1.5, blur, -0.5, 0)
cv2.imwrite("preview_sharp.png", sharp)
```

Expected Output: Cameraman image with slightly enhanced edges — buildings and figures look crisper.

Example 3: Demonstrate noise amplification

```
import cv2, numpy as np
from skimage.data import camera
```

```

img = camera()
np.random.seed(0)
noisy = np.clip(img + np.random.normal(0, 10, img.shape), 0, 255).astype(np.uint8)

blur = cv2.GaussianBlur(noisy, (0, 0), 1.5)
sharp = cv2.addWeighted(noisy, 1.5, blur, -0.5, 0)

cv2.imwrite("noise_sharp.png", sharp)
print("noisy std :", noisy.std())
print("sharp std :", sharp.std())

```

Expected Output: The "sharp" version has a noticeably higher std (more contrast everywhere — including noise). The image visually looks grainier than noisy.

4. Parameter Sensitivity

(Lessons 2-5 cover specific parameters per technique.)

5. Common Mistakes

Mistake	What Happens	Fix
Sharpening a noisy image first	Noise amplified	Denoise first (M7), then sharpen lightly
Sharpening more than needed	Halos / ringing	Use small amount values
Sharpening already-sharp images	Ugly artefacts	Don't sharpen for the sake of it

6. Practice Tasks

- Compute edge_energy for camera() and a 2σ -blurred copy. Note the difference.
- Apply a light sharpen with addWeighted(img, 1.3, blur, -0.3, 0).
- Add Gaussian noise ($\sigma=10$) before sharpening; observe the noise amplification.

7. Key Takeaways

- Sharpening = original + amount $\times$ (original – blurred).
- Edges have high gradient; sharpening boosts them.
- Noise has high gradient too — sharpening amplifies it.
- Always denoise first if the image is noisy.

8. What's Next

The Laplacian — the simplest second-derivative operator behind sharpening.

Lesson 2: Laplacian Kernel

Module: 8 — Sharpening

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Use `cv2.Laplacian` to compute the second derivative.
- Apply a Laplacian-based sharpening kernel directly.
- Recognize the sign of the Laplacian as the "edge" indicator.

Prerequisites

- Module 7 (convolution), Module 8 Lesson 1.

1. Why This Matters

The Laplacian is the canonical second-derivative operator in image processing. It responds to all edges, regardless of orientation. Many sharpening kernels are just "identity – Laplacian-ish" — knowing the Laplacian unlocks both.

2. Concept

2.1 The math

The Laplacian is the sum of second partial derivatives:

$$\nabla^2 I = \partial^2 I / \partial x^2 + \partial^2 I / \partial y^2$$

For a 3×3 discrete approximation:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad \text{OR (8-conn)} \quad \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Both kernels respond to rapid intensity changes. In flat regions the output is ~0; at edges it spikes positive or negative.

2.2 The OpenCV call

```
lap = cv2.Laplacian(img, cv2.CV_64F, ksize=3)
```

- `ddepth = CV_64F` or `CV_32F` is required — the Laplacian can be negative.
- `ksize=1` uses the 4-connected kernel; larger `ksize` uses Sobel-derived approximations.

2.3 Visualising the Laplacian

```
lap_vis = cv2.convertScaleAbs(lap) # |lap|, clipped to uint8
```

This collapses negative values to positive — good for visualisation, but loses the sign information you need for sharpening.

2.4 Sharpening via Laplacian

$\text{sharpened} = \text{original} - \text{amount} \times \text{Laplacian}$

Because the Laplacian is positive on the dark side of an edge and negative on the bright side, subtracting it pushes edges further apart — making the bright side brighter and the dark side darker.

A common one-shot kernel:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

That's identity (centre=1) – Laplacian (centre=-4) = centre 5, edges -1, scaled. Sums to 1 (preserves brightness).

2.5 Pre-blur to avoid noise

The Laplacian amplifies anything with a high second derivative — including noise. Standard practice: blur slightly first (Gaussian $\sigma \sim 1.0$).

2.6 LoG — Laplacian of Gaussian

Gaussian blur → Laplacian is one common edge / blob detector (preview of M9 / M13). Saves work because Gaussian blur is separable.

3. Code Examples

Example 1: Laplacian visualisation

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
lap = cv2.Laplacian(img, cv2.CV_64F, ksize=3)
lap_vis = cv2.convertScaleAbs(lap)
cv2.imwrite("laplacian.png", lap_vis)
print("lap range:", lap.min(), lap.max())
```

Expected Output: laplacian.png highlights edges — building outlines, the cameraman's silhouette — bright on black background. Console prints negative min, positive max (e.g. -120 to 90).

Example 2: Sharpening via single kernel

```
import cv2, numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
k = np.array([[ 0, -1,  0],
              [-1,  5, -1],
              [ 0, -1,  0]], dtype=np.float32)
```

```
sharp = cv2.filter2D(img, -1, k)
cv2.imwrite("coffee_lap_sharp.png", sharp)
```

Expected Output: Coffee image with crisper edges — bean outlines and cup rim look sharper.

Example 3: Pre-blur + Laplacian for cleaner sharpening

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
blur = cv2.GaussianBlur(img, (0, 0), 1.0)
lap = cv2.Laplacian(blur, cv2.CV_64F, ksize=3)

# sharpened = original - amount * lap
sharp = np.clip(img.astype(np.float32) - 0.7 * lap, 0, 255).astype(np.uint8)
cv2.imwrite("camera_lap_blur_sharp.png", sharp)
```

Expected Output: Cameraman with enhanced edges; less noisy than a no-blur Laplacian sharpen.

Example 4: Amount tuning

```
import cv2, numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR).astype(np.float32)
blur = cv2.GaussianBlur(img, (0, 0), 1.0)
lap = cv2.Laplacian(blur, cv2.CV_32F, ksize=3)

for amt in [0.3, 0.7, 1.5]:
    sharp = np.clip(img - amt * lap, 0, 255).astype(np.uint8)
    cv2.imwrite(f"coffee_amt_{amt}.png", sharp)
```

Expected Output: Three saved files. 0.3 is subtle. 1.5 over-sharpens with visible halos.

4. Parameter Sensitivity

Param	Effect
amount (subtracted)	larger = sharper, more haloes
Pre-blur σ	larger = cleaner sharpen, less detail captured
ksize in cv2.Laplacian	larger = smoother / more spread response

5. Common Mistakes

Mistake	What Happens	Fix
ddepth=-1 to Laplacian	Negative values wrap	Use CV_64F / CV_32F
Skip pre-blur	Noise amplified	Light Gaussian first
Visualise Laplacian directly	All-black image (signed values mostly small)	Use cv2.convertScaleAbs
Apply sharpen multiple times	Heavy halos	One pass with right amount

6. Practice Tasks

- Compute the Laplacian of camera(). Visualise with convertScaleAbs.

- Apply the 3×3 sharpening kernel $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$ to `coffee()`.
- Tune amount $\in \{0.3, 0.7, 1.5\}$ in the "original – amount × Laplacian" formulation.

7. Key Takeaways

- Laplacian = sum of second derivatives; responds to all edges.
- Use `CV_64F` to avoid negative-value wrap.
- original – amount × Laplacian = sharpening recipe.
- Pre-blur slightly to avoid noise amplification.

8. What's Next

Unsharp masking — the historical, gentler way to sharpen.

Lesson 3: Unsharp Masking

Module: 8 — Sharpening

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Implement unsharp masking by hand with `cv2.GaussianBlur` + `cv2.addWeighted`.
- Tune amount, radius, threshold like Photoshop's USM dialog.
- Understand the darkroom origin (and the confusing name).

Prerequisites

- Module 8 Lesson 2

1. Why This Matters

Unsharp Masking (USM) is the standard sharpening method in Photoshop, Lightroom, GIMP, RawTherapee — everywhere. It produces controllable, photographer-friendly results. The name is confusing: it does not mean "remove sharpness" — it means "sharpen using an UNSHARP (blurred) mask".

2. Concept

2.1 The darkroom origin

In film printing, photographers placed a slightly out-of-focus negative copy ("unsharp mask") on top of the original during exposure to subtract low-frequency content — leaving the high frequencies (edges) more visible in the print. Digital USM mimics this.

2.2 The formula

$$\text{sharpened} = \text{original} + \text{amount} \times (\text{original} - \text{blurred})$$

Where `blurred` = `GaussianBlur(original, radius)`.

The middle term (`original - blurred`) is a high-pass image — small in flat regions, large at edges. Multiply by amount, add back to the original → edges exaggerated.

2.3 In code (one line)

```
blur = cv2.GaussianBlur(img, (0, 0), radius)
sharp = cv2.addWeighted(img, 1 + amount, blur, -amount, 0)
```

Equivalent algebra:

$$\begin{aligned} \text{sharp} &= (1 + \text{amount}) \times \text{original} - \text{amount} \times \text{blurred} \\ &= \text{original} + \text{amount} \times (\text{original} - \text{blurred}) \end{aligned}$$

2.4 The three knobs (Photoshop USM)

Param	Effect
Amount	How strong (typical 0.5–2.0)
Radius (Gaussian σ)	Edge halo width: small = fine detail, large = big features
Threshold	Only sharpen where local contrast exceeds this — prevents amplifying flat-area noise

2.5 Threshold implementation

```
diff = img.astype(np.int16) - blur.astype(np.int16)
mask = np.abs(diff) > threshold
sharp = img.copy()
sharp[mask] = np.clip(img[mask] + amount * diff[mask], 0, 255).astype(np.uint8)
```

Below the threshold, pixels are left untouched — the smooth sky stays smooth.

2.6 Color images

Apply USM to the luminance channel only (Y of YCrCb or L of LAB) to avoid colour shifts:

```
ycc = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)
Y = ycc[..., 0]
blur_Y = cv2.GaussianBlur(Y, (0, 0), radius)
ycc[..., 0] = cv2.addWeighted(Y, 1 + amount, blur_Y, -amount, 0)
out = cv2.cvtColor(ycc, cv2.COLOR_YCrCb2BGR)
```

3. Code Examples

Example 1: Basic USM

```
import cv2
from skimage.data import camera

img = camera()
blur = cv2.GaussianBlur(img, (0, 0), sigmaX=1.5)
sharp = cv2.addWeighted(img, 1.5, blur, -0.5, 0)
cv2.imwrite("camera_usm.png", sharp)
```

Expected Output: Cameraman image with crisper edges (building outlines, figure silhouette pop more) compared to original.

Example 2: Tune the three knobs

```
import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
for radius in [0.8, 2.0, 5.0]:
    for amount in [0.5, 1.5]:
        blur = cv2.GaussianBlur(img, (0, 0), radius)
        out = cv2.addWeighted(img, 1 + amount, blur, -amount, 0)
        cv2.imwrite(f"coffee_r{radius}_a{amount}.png", out)
```

Expected Output: 6 files. Larger radius produces wider halos. Larger amount produces stronger contrast.

Example 3: With threshold

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
blur = cv2.GaussianBlur(img, (0, 0), 1.5)
amount, threshold = 1.0, 8

diff = img.astype(np.int16) - blur.astype(np.int16)
mask = np.abs(diff) > threshold
sharp = img.copy()
sharp[mask] = np.clip(img[mask].astype(np.int16) + (amount *
diff[mask]).astype(np.int16), 0, 255).astype(np.uint8)
cv2.imwrite("camera_usm_thresh.png", sharp)
print("sharpened pixels:", mask.sum(), "out of", mask.size)
```

Expected Output: Same sharpening on edges but the sky stays smooth (no grain amplification). Console prints something like sharpened pixels: 30000 out of 262144.

Example 4: Color-safe USM via Y channel

```
import cv2
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
ycc = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)
Y = ycc[..., 0]
blur_Y = cv2.GaussianBlur(Y, (0, 0), 1.5)
ycc[..., 0] = cv2.addWeighted(Y, 1.5, blur_Y, -0.5, 0)
out = cv2.cvtColor(ycc, cv2.COLOR_YCrCb2BGR)
cv2.imwrite("coffee_usm_y.png", out)
```

Expected Output: Coffee image with sharper texture; colors not shifted (compare to per-channel USM, which can saturate colors weirdly).

4. Parameter Sensitivity

Param	Lower	Higher
amount	barely visible	aggressive, halos
radius (σ)	sharpens fine detail	broad halos around big features
threshold	sharpens everywhere (noise too)	sharpens only strong edges

Typical presets:

- "Web photo sharpen": amount=0.5, radius=1, threshold=3
- "Print sharpen": amount=1.0-1.5, radius=1-2, threshold=0
- "Creative crunch": amount=2, radius=4

5. Common Mistakes

Mistake	What Happens	Fix
USM on noisy image	Noise amplified	Denoise (M7) first or use threshold
Per-channel BGR USM	Color shifts	Apply on Y of YCrCb
Skipping clip	Wraparound	cv2.addWeighted clips automatically
radius too large	Soft, halo'd result	Start with 1-2

6. Practice Tasks

- Apply USM with amount=1.5, radius=1.5 to camera(). Save.
- Compare radius=0.5 vs radius=5 — note the halo width.
- Implement threshold-based USM and apply with threshold=8 on coffee().

7. Key Takeaways

- $USM = original + amount \times (original - blurred)$.
- Three knobs: amount, radius (σ), threshold.
- Use Y channel for color to avoid hue shifts.
- "Unsharp" refers to the mask, not the result.

8. What's Next

High-pass filtering — the frequency-domain view of what USM is doing.

Lesson 4: High-Pass Filtering

Module: 8 — Sharpening

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Construct a high-pass image as original – blurred.
- Visualise the high-pass content of an image.
- Apply a high-pass kernel directly with `cv2.filter2D`.

Prerequisites

- Module 8 Lesson 3

1. Why This Matters

Sharpening, edge detection, and many forensics tasks all start with isolating the high-frequency content. A low-pass filter (blur) keeps slow changes; subtract it from the original and you have only the fast changes.

2. Concept

2.1 Frequency picture

- Low frequencies = slowly varying intensity (smooth regions, gradients).
- High frequencies = rapidly varying intensity (edges, fine texture, noise).

A Gaussian blur is a low-pass filter — it keeps the lows, attenuates the highs.

2.2 Building a high-pass

```
high_pass = original - blurred
```

The output centres around 0 — flat regions are near-zero, edges are large positive or negative.

To visualise it as an image, add an offset (e.g. 128) so 0 → mid-gray:

```
hp_vis = ((img.astype(np.int16) - blur.astype(np.int16)) + 128).clip(0, 255).astype(np.uint8)
```

2.3 Direct high-pass kernel

A 3×3 high-pass that approximates "subtract average":

```
[ -1  -1  -1 ]
[ -1   8  -1 ]
[ -1  -1  -1 ]
```

Sums to 0 → output near zero in flat regions; nonzero at edges. Pure high-pass.

2.4 Sharpening via high-pass

```
sharpened = original + amount × high_pass
```

Identical to USM — just stating it in frequency-domain language.

2.5 Frequency-domain preview

In Module 14 (DFT) you'll do the same thing via the Fourier transform — explicitly multiplying frequency coefficients. The spatial-domain methods here are mathematically equivalent for symmetric kernels, faster, and easier to reason about for everyday work.

3. Code Examples

Example 1: Visualise high-pass content

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
blur = cv2.GaussianBlur(img, (0, 0), 2.0)
hp = (img.astype(np.int16) - blur.astype(np.int16))
hp_vis = (hp + 128).clip(0, 255).astype(np.uint8)
cv2.imwrite("camera_highpass.png", hp_vis)
print("hp range:", hp.min(), hp.max(), " mean ~ 0?", round(hp.mean(), 2))
```

Expected Output: camera_highpass.png is mid-gray everywhere with edges showing as bright/dark traces. Console prints range like -90 80 and a mean very close to 0.

Example 2: Direct high-pass kernel

```
import cv2, numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
k = np.array([[ -1, -1, -1],
              [ -1,  8, -1],
              [ -1, -1, -1]], dtype=np.float32)
hp = cv2.filter2D(img, cv2.CV_32F, k)
hp_vis = cv2.convertScaleAbs(hp)
cv2.imwrite("coffee_hp_kernel.png", hp_vis)
```

Expected Output: coffee_hp_kernel.png shows only the edges and fine texture — mostly black with bright outlines.

Example 3: Sharpen by adding high-pass back

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
blur = cv2.GaussianBlur(img, (0, 0), 1.5)
hp = img.astype(np.int16) - blur.astype(np.int16)
sharp = np.clip(img.astype(np.int16) + (1.0 * hp), 0, 255).astype(np.uint8)
cv2.imwrite("camera_hp_sharp.png", sharp)
```

Expected Output: Same kind of sharpening as USM — confirms the equivalence.

4. Parameter Sensitivity

Param	Effect
Blur radius σ	smaller σ = finer-detail high-pass; bigger σ = broader features in high-pass
Kernel "centre" weight	controls amplitude — 8 vs 4 etc.

5. Common Mistakes

Mistake	What Happens	Fix
Storing high-pass as uint8	Negative parts wrap	Use int16 / float and shift for display
Sum of kernel $\neq 0$	Constant offset in output	Make sure rows sum to 0 for pure high-pass
Multiplying high-pass too much	Halos	Use small amounts (0.5-1.5)

6. Practice Tasks

- Build a high-pass of camera() with $\sigma=2.0$; visualise with +128 offset.
- Apply the direct 3×3 high-pass kernel to coffee() (cv2.CV_32F output).
- Sharpen using original + high_pass; confirm identical to USM with amount=1.

7. Key Takeaways

- High-pass = original – blurred.
- Centres around 0 in flat regions; large at edges.
- Sharpening = add scaled high-pass back to original.
- Direct kernels summing to 0 work too.

8. What's Next

Custom kernels — designing your own sharpening kernels for special effects.

Lesson 5: Custom Sharpening Kernels

Module: 8 — Sharpening

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Design custom sharpening kernels.
- Use emboss-style and edge-enhance kernels.
- Tune kernel sum to control brightness preservation.

Prerequisites

- Module 8 Lesson 4

1. Why This Matters

The Photoshop "smart sharpen", many camera in-body sharpening profiles, and emboss-like artistic effects all come from custom kernels. Knowing the structure lets you read any kernel and predict what it does.

2. Concept

2.1 Kernel anatomy

A sharpening kernel always has:

- A large positive centre weight (boosts the current pixel).
- Negative weights around it (subtracts neighbours).
- Sum ≈ 1 (or 0, with a bias added later) — preserves brightness.

2.2 Classic sharpening (4-neighbour Laplacian + identity)

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Sums to 1. Standard "sharpen" filter.

2.3 Strong sharpening (8-neighbour)

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Sums to 1. More aggressive — affects diagonals too.

2.4 Soft sharpening (less aggressive)

$$\begin{bmatrix} 0 & -0.5 & 0 \\ -0.5 & 3 & -0.5 \\ 0 & -0.5 & 0 \end{bmatrix}$$

Sums to 1. Centre weight only 3 (vs 5) — subtler.

2.5 Emboss (artistic effect)

```
[ -2  -1   0 ]
[ -1   1   1 ]
[  0   1   2 ]
```

Sums to 1. Produces a "3D embossed" look — directional sharpening that makes the image look like raised metal. Often add +128 bias and clip to keep mid-gray neutral.

2.6 Brightness preservation

If kernel sum = 1, average brightness is preserved. If sum > 1 → image becomes brighter. If sum < 1 → image becomes darker. If sum = 0 (pure high-pass) → output centred on zero; add a 128 offset for display.

2.7 Designing your own

Start from a "stronger" template (centre 5 or 9), then adjust:

- Reduce centre + edge magnitudes proportionally for subtler effect.
- Skew weights asymmetrically for directional effects (like emboss).
- Always check the sum.

3. Code Examples

Example 1: Classic 5-kernel

```
import cv2, numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
k = np.array([[ 0, -1,  0],
              [-1,  5, -1],
              [ 0, -1,  0]], dtype=np.float32)
print("sum =", k.sum())
sharp = cv2.filter2D(img, -1, k)
cv2.imwrite("coffee_classic.png", sharp)
```

Expected Output: sum = 1.0 and coffee_classic.png is the coffee image with crisper edges.

Example 2: Strong 9-kernel

```
import cv2, numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
k = np.array([[-1, -1, -1],
              [-1,  9, -1],
              [-1, -1, -1]], dtype=np.float32)
sharp = cv2.filter2D(img, -1, k)
cv2.imwrite("coffee_strong.png", sharp)
```

Expected Output: Coffee with more aggressive sharpening — visible halos on the cup rim.

Example 3: Emboss

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
k = np.array([[ -2, -1, 0],
              [-1,  1, 1],
              [ 0,  1, 2]], dtype=np.float32)
emb = cv2.filter2D(img, cv2.CV_32F, k) + 128
emb_vis = np.clip(emb, 0, 255).astype(np.uint8)
cv2.imwrite("camera_emboss.png", emb_vis)
```

Expected Output: Cameraman appearing as a 3D-stamped relief on a mid-gray background — top-left edges dark, bottom-right edges bright.

Example 4: Custom soft sharpen

```
import cv2, numpy as np
from skimage.data import coffee

img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
k = np.array([[ 0, -0.5,  0],
              [-0.5, 3, -0.5],
              [ 0, -0.5,  0]], dtype=np.float32)
print("sum =", k.sum())
out = cv2.filter2D(img, -1, k)
cv2.imwrite("coffee_soft.png", out)
```

Expected Output: sum = 1.0. coffee_soft.png is more subtly sharper than the classic kernel.

4. Parameter Sensitivity

Kernel "strength" (centre weight)	Effect
centre = 3	very subtle
centre = 5	classic / good default
centre = 9	strong — risk of halos
centre = 1 (emboss)	directional / artistic

5. Common Mistakes

Mistake	What Happens	Fix
Kernel sum > 1	image brightens overall	rescale or accept brightness shift
Forgetting CV_32F for negative-sum kernels	clipping at 0 wipes out shape	use float, add bias, clip at end
Designing kernels without checking sum	unexpected brightness/contrast shift	always print k.sum()

6. Practice Tasks

- Apply the classic, strong, and soft sharpen kernels to coffee(). Compare.
- Apply emboss kernel to camera(). Add +128 bias.

- Design a "vertical-only" sharpen kernel; explain when it might be useful.

7. Key Takeaways

- Sharpening kernels: positive centre, negative neighbours.
- Sum to 1 = preserves brightness.
- Stronger centre = more sharpening = more halo risk.
- Asymmetric kernels (emboss) give artistic 3D effects.

8. What's Next

End of Module 8. Next: Module 9 — Edge Detection — where the high-pass ideas you've built around get formalised.

Module 8 — Exercises

15 exercises.

8.1 Why

- (E) Compute `edge_energy` for `camera()` and $\sigma=2$ blurred copy.
- (M) Apply `USM(amount=1.3,  $\sigma=1.5$ )` to `camera()`.
- (H) Add Gaussian noise $\sigma=10$; sharpen; show noise amplification.

8.2 Laplacian

- (E) Compute `cv2.Laplacian(camera, CV_64F, ksize=3)`. Visualise.
- (M) Apply 3×3 sharpen kernel $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$ to `coffee()`.
- (H) Tune amount $\in \{0.3, 0.7, 1.5\}$ in "`img - amount  $\times$  Laplacian`".

8.3 USM

- (E) USM `amount=1.5, radius=1.5` on `camera()`.
- (M) Compare radius 0.5 vs 5 — note halo width.
- (H) Add threshold-based USM (only sharpen where $|\text{diff}| > 8$).

8.4 High-pass

- (E) Compute $\sigma=2$ high-pass of `camera()`; visualise with +128 offset.
- (M) Apply direct 3×3 high-pass kernel to `coffee()`.
- (H) Sharpen by adding scaled high-pass to original; verify same as `USM(amount=1)`.

8.5 Custom kernels

- (E) Apply classic, strong, soft kernels to `coffee()`; compare.
- (M) Apply emboss kernel to `camera()` (+128 bias).
- (H) Design a vertical-only sharpening kernel; describe its use.

Module 8 — Quiz

10 MCQs.

Q1

Sharpening = original + amount $\times$? A. blurred B. original C. (original – blurred) D. Laplacian²

Answer: C (L1)

Q2

The Laplacian is the: A. first derivative B. second derivative sum C. inverse D. mean

Answer: B (L2)

Q3

Why use CV_64F (not -1) for Laplacian output? A. higher precision B. Laplacian can be negative — uint8 wraps C. faster D. required by API

Answer: B (L2)

Q4

"Unsharp Mask" name comes from: A. removing sharpness B. using a blurred (unsharp) mask in the formula C. raw photo metadata D. RAW format

Answer: B (L3)

Q5

USM amount=1.0 with radius=1.5 produces: A. heavy halos B. subtle edge crispness C. blurry image D. negative image

Answer: B (L3)

Q6

For color USM, apply to: A. each BGR channel B. only B C. Y of YCrCb or L of LAB D. RGB after conversion

Answer: C (L3)

Q7

A 3 $\times$ 3 high-pass kernel sums to: A. 1 B. 0 C. 255 D. depends

Answer: B (L4)

Q8

A classic sharpen kernel has centre value: A. 1 B. 5 C. 0 D. 255

Answer: B (L5)

Q9

The strong 8-neighbour kernel $\begin{bmatrix} -1 & -1 & -1 \\ -1 & 9 & -1 \\ -1 & -1 & -1 \end{bmatrix}$ sums to: A. 9 B. 1 C. 0 D. -8

Answer: B (L5)

Q10

Sharpening a noisy image: A. removes noise B. amplifies noise C. inverts noise D. has no effect on noise

Answer: B (L1)

Module 8 — Assignment: Smart Sharpener

Estimated time: 1.5-2 hours

Combines: Module 8 + Modules 6-7

Brief

Build a CLI smart sharpener that:

- Optionally denoises first.
- Applies USM with a noise-aware threshold.
- Reports before/after edge-energy.

Requirements

- sharpen.py INPUT [--output OUT] [--amount 1.0] [--radius 1.5] [--threshold 5] [--denoise]
- If --denoise: pass through cv2.bilateralFilter(d=7, sigmaColor=50, sigmaSpace=50) first.
- USM on Y of YCrCb (color-safe).
- Threshold: only sharpen where local diff exceeds threshold.
- Print mean edge energy before/after.
- Save result.

Constraints

- Modules 1-8 only.
- USM applied via the standard addWeighted formula.

Expected Output

```
$ python sharpen.py portrait.jpg --amount 1.2 --radius 1.5 --threshold 6 --denoise
denoised:    edge energy 12.4 -> 9.1
sharpened:   edge energy 9.1 -> 15.8
saved -> portrait_sharp.jpg
```

Grading Rubric

Criterion	Weight
USM on Y channel	25%
Threshold logic	20%
Denoise pass	15%
Edge energy reporting	15%
Argparse + style	25%

Submission

Save as assignment_module8.py.

Module 8 — Mini Project: Photoshop-Style USM Tool

Overview

Replicate Photoshop's "Unsharp Mask" dialog with three trackbars: amount, radius, threshold. Real-time preview alongside the original.

Concepts Used

- USM (M8 L3)
- Threshold-only sharpen (M8 L3 advanced)
- Trackbars (M2 L3)

Features

- 3 trackbars: amount ($\times 10$), radius ($\times 10 = \sigma$), threshold.
- Side-by-side: original | sharpened.
- s saves; q quits.

Starter Code

usm_tool.py:

```
import sys, cv2, numpy as np

def usm_threshold(img, amount, radius, threshold):
    if img.ndim == 3:
        ycc = cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb)
        Y = ycc[..., 0]
        Y_sharp = usm_threshold(Y, amount, radius, threshold)
        ycc[..., 0] = Y_sharp
        return cv2.cvtColor(ycc, cv2.COLOR_YCrCb2BGR)
    blur = cv2.GaussianBlur(img, (0, 0), max(0.1, radius))
    diff = img.astype(np.int16) - blur.astype(np.int16)
    mask = np.abs(diff) > threshold
    out = img.copy().astype(np.int16)
    out[mask] = np.clip(out[mask] + (amount * diff[mask])).astype(np.int16), 0, 255)
    return out.astype(np.uint8)

def run(path):
    img = cv2.imread(path)
    if img is None: raise FileNotFoundError(path)
    if max(img.shape) > 700:
        img = cv2.resize(img, None, fx=0.5, fy=0.5)

    cv2.namedWindow("usm")
    cv2.createTrackbar("amount x10", "usm", 10, 30, lambda v: None)
    cv2.createTrackbar("radius x10", "usm", 15, 50, lambda v: None)
    cv2.createTrackbar("threshold", "usm", 0, 50, lambda v: None)

    while True:
        a = cv2.getTrackbarPos("amount x10", "usm") / 10
        r = cv2.getTrackbarPos("radius x10", "usm") / 10
```

```

th = cv2.getTrackbarPos("threshold", "usm")
out = usm_threshold(img, a, r, th)
combo = np.hstack([img, out])
cv2.putText(combo, f"a={a} r={r} th={th}", (10, 25),
              cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
cv2.imshow("usm", combo)
k = cv2.waitKey(30) & 0xFF
if k == ord('q'): break
if k == ord('s'): cv2.imwrite("usm_out.png", out); print("saved
usm_out.png")
cv2.destroyAllWindows()

if __name__ == "__main__":
    run(sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/coffee.png")

```

Expected Interaction

Move sliders; the sharpened panel updates in real time. Halos appear when amount > 2.0 or radius > 3.

Extension Challenges

- Add a "luminance only" toggle (already implemented; expose as flag).
- Show the high-pass mask in a third panel.
- Add a "before/after" toggle key.

Grading Rubric

Criterion	Weight
Three trackbars work	25%
USM math correct	25%
Threshold path	20%
Side-by-side display	15%
Style	15%